



Web Development Foundations

Week 6 - Debugging: "Things Break - And I Can Fix Them"

Day 2: "Debugging Multiple Issues"

1



Reconnect

Monday:

- reproduced one problem
- read one console clue
- compared HTML and JavaScript
- fixed one selector
- verified the result

Today:

- we keep that process when more than one issue appears.

2



From One Bug to a Small Issue List

Keep the same loop—apply it to each issue.

Monday: One Issue

Current Issue

selector mismatch

Next: A Small Issue List

Issue List (in order)

- 1 selector mismatch
- 2 condition error
- 3 CSS mismatch

One issue at a time. Same process from start to finish.

Same process. More than one issue.

- Work the list in order.
- Complete the loop for each issue.
- Keep your evidence and explanations.

One Bug To Issue List

3



The Working Problem: More Than One Thing Can Be Wrong

A page can have multiple issues:

- JavaScript selector mismatch
- condition logic error
- CSS selector mismatch
- confusing user feedback

Fixing one issue may reveal the next.

That is normal.

4



Multiple Layered Issues on One Page

More than one thing can be the problem.

Layered Issues

- 1 JS: JavaScript selector mismatch. The script cannot find the element. Nothing happens when the button is clicked.
- 2 Condition logic error: After the selector is fixed, the logic prevents the message from showing.
- 3 CSS: CSS selector mismatch. After the message appears, the style does not apply because the CSS selector is wrong.

Fixing one issue may reveal the next.

- Work step by step.
- Verify after each fix.
- Be ready for what's next.

Multiple Issue Stack

5



What Counts as Success Today

- you choose one likely cause first
- you make one focused change
- you retest before moving on
- you document what happened
- at least two fixes are verified

6

Prioritize The First Cause

Start with the issue that blocks testing.

Good first checks:

- ▶ script file linked?
- ▶ console error present?
- ▶ selector matches HTML?
- ▶ event listener connected?

Cosmetic issues can wait until behavior works.





Debugging Priority

Check in order. One step at a time.

1



script linked?

Is the JavaScript file connected to the page?

2



console error?

Does the console show an error that stops code?

3



selector matches?

Does the selector match the HTML element?

4



event connected?

Is the event listener attached correctly?

5



condition logic?

Is the logic allowing the code to run as expected?

6



styling issue?

Does the styling affect what you see?



Start with what blocks testing.

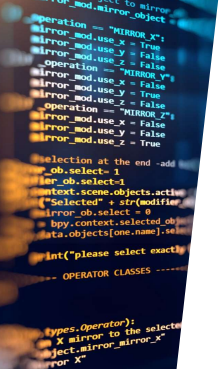
 Fix blocking issues first.

 Verify after each change.

 Move down the list.



Priority Ladder



```

// x, y, z = modifier, ob =
// mirror object to mirror
// mirror, mod, mirror object

function Mirror(x, y, z) {
  operation = "MIRROR X";
  mirror.mod.use.x = true;
  mirror.mod.use.y = false;
  operation = "MIRROR Y";
  mirror.mod.use.x = false;
  mirror.mod.use.y = true;
  operation = "MIRROR Z";
  mirror.mod.use.x = false;
  mirror.mod.use.y = false;
  mirror.mod.use.z = true;

  selection at the end - add
  let_ob.select-1
  let_ob.select-1
  context.scene.objects.active
  ["select"] = selectedOb.name;
  mirror_ob.select = 0
  bpy.context.selected_objects
  data.objects[one.name]=selectedOb
  print("please select exactly one object")
}

// OPERATOR CLASSES -----

// Mirror class
// types.Operator():
// x mirror to the selected object
// select_mirror_mirror_x
// mirror X

// context:
// next_active_object is not

```



What We Will Use Today

- ▶ [issue list](#)
- ▶ [priority](#)
- ▶ [console error](#)
- ▶ [`console.log\(\)`](#)
- ▶ [selector check](#)
- ▶ [condition check](#)
- ▶ [CSS mismatch check](#)
- ▶ [verification note](#)

Today's Multi-Issue Debugging Toolbox

Use the right tools in the right order.

1. Issue List
Write down what you see, both on and off the machine.

2. Strategy
Check it, make a plan, start with the obvious, work your way through.

3. Connect Data
Look for errors that may connect.

4. Research Tool
Add up to see what's out there and how.

5. Questioning
Generate the questions, research the answers, the code to run.

6. Creation
Verify the hypothesis, the code to run.

7. CSB Hypothesis Check
Check the hypothesis, the code to run.

8. Verification
Repeat until the hypothesis is correct, the code to run.

Use the tools. Follow the priority. Verify the fix.

Same process. Different tools.

Identify → Prioritize → Investigate → Fix → Verify → Exploit

Verification tools:
☒ Windows
☒ Metasploit
☒ Git
☒ Kali Linux
☒ Netcat

Do Not Fix Everything At Once

Avoid This

- ▶ change selector
- ▶ change condition
- ▶ change CSS
- ▶ refresh once
- ▶ hope it works

Use This

- ▶ change one likely cause
- ▶ retest
- ▶ record result
- ▶ then choose the next issue



Debug Smarter, Not Harder

Two ways to handle multiple issues.



Chaotic Approach

Change Everything



Hard to know what fixed it.
New problems can appear.

Calm Approach

One Fix at a Time

- change one cause**
Pick the most recent (easiest) issue and change only that.
- reset**
Run the same test or action.
- record result**
What changed? What stayed the same?
- choose next issue**
Use your results to decide what to tackle next.

Clear results. Steady progress.
Fewer new problems.



Retesting tells you what worked.



Small, targeted changes.

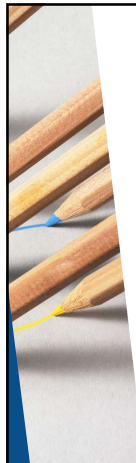


Clear evidence from each step.



Confident fixes, one at a time.

One Fix At A Time



Console Logs As Evidence

`console.log()` can answer:

- ▶ did this function run?
- ▶ what value did the input contain?
- ▶ which branch did the condition choose?
- ▶ did the code reach this line?

Use logs to test a question.
Then remove or clean them later.

13



console.log as Evidence Checkpoints

Place logs at key points to prove what your code is doing.

Simple Code Path

```

</> User clicks button
↓
function started? Did the function run?
f() handleClick() function starts
↓
Read input value Read the text box
↓
Check condition Choose a branch
↓
Show result Update the message
↓
These logs are checkpoints, not final answers. They help you see what is happening.

```

Console Output (Simplified)

- 1 function started
handleClick() running
- 2 input value read
username = "12"
- 3 condition branch chosen
branch = "high score"

Each message answers the question at that step.

Use logs to test a question.

Did this run? What value did we get? Which branch did we take? Record what you learn.

14



Proper `console.log()` Format

Use logs that explain what you are checking.

Less useful:

- ▶ `console.log(taskText);`

More useful:

- ▶ `console.log("Task text:", taskText);`

Best habit:

- ▶ - label the value
- ▶ - log one question at a time
- ▶ - remove extra logs after debugging (or comment them out)

15



Proper Beginner console.log Formatting

Make your logs easy to read and easy to understand.

Less Useful

```
console.log(taskText);
```

What am I looking at? What question does this answer?

Output (example): Read the task

More Useful

```
console.log("Task text:", taskText);
```

Clear label. Clear purpose. Now I know what this value means.

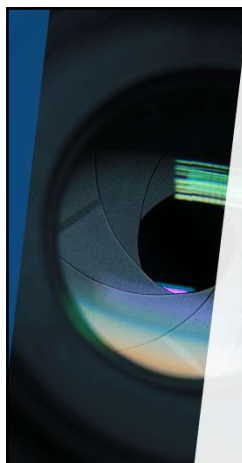
Output (example): Task text: Read the task

Three Good Logging Habits

1. Label the value
Add a short label so future you knows what it is.
2. Log one question at a time
Keep each log focused on one thing.
3. Remove extra logs later
Clean up logs once the issue is solved.

A useful log explains what you are checking. Logs help you see the program's behavior and guide your next steps.

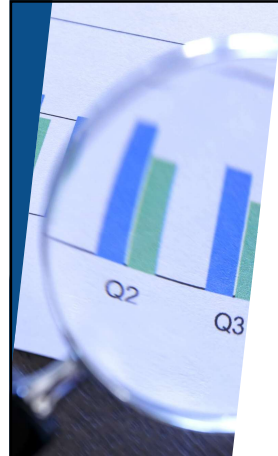
16



Demo

"Multi-Issue Debugging"

17

What to Watch For

- ▶ first visible symptom
- ▶ first console clue
- ▶ selector mismatch
- ▶ retest after selector fix
- ▶ condition error
- ▶ CSS issue handled after behavior works

18

Multi-Issue Debugging (Recorded Demo)
One page. Three issues. One methodical path.

Task Page

Score Checker
Enter a score and click Check.
\$5
Check Score

Result
(nothing shown)

Three Issues on This Page

1. Selector information: Select can't find the button or modal area.
2. Condition error: This page prevents the modal from showing.
3. CSS mismatch: The result text appears but has no style.

Methodical Path (One Issue at a Time)

1. fix selector: Correct the selector so elements can be found.
2. retest: Run the same action. Confirm new behavior.
3. fix condition: Correct the logic to the result can show.
4. retest: Run the same action. Confirm new behavior.
5. check styling: Fix the CSS selector or style so it looks right.
6. retest: Run the same action. Confirm new behavior.

What You'll See in the Demo

- Small, focused fixes.
- Retest after each fix.
- Clear progress.
- Some progress, three issues.

Order matters.

Fix the cause that blocks testing first. Retest to see what changed. Use the results to choose the next issue. Slowly work your way to a working, well-styled page.

Demo: Multi-Issue Debugging

19

Retest After Each Fix

After each change, ask:

- ▶ Did the original symptom change?
- ▶ Did the console error change?
- ▶ Did a new issue appear?
- ▶ Is the current fix verified?
- ▶ What is the next smallest check?

Retesting turns edits into evidence.

20

Retesting After Each Fix
Small change. Retest. Collect evidence.

1. Fix 1: Make one targeted change. → Retest: Run the same action or test again. → Evidence: Record what happened and what changed.
2. Fix 2: Make one targeted change. → Retest: Run the same action or test again. → Evidence: Record what happened and what changed.
3. Fix 3: Make one targeted change. → Retest: Run the same action or test again. → Evidence: Record what happened and what changed.

Retesting turns edits into evidence.

Change one issue. Retest the same action. Record what you see. Use results to choose the next issue.

Retest After Each Fix

21

A Good AI Debugging Prompt

A useful AI debugging prompt includes:

- ▶ observed symptom
- ▶ exact console error
- ▶ small relevant code section
- ▶ what you already checked
- ▶ what kind of help you want

Ask for explanation and possible causes.
Do not ask for a replacement project.

22

The Shape of a Good AI Debugging Prompt
Clear inputs help you get useful, focused answers.

Your Prompt to AI

1. Observed Symptom: What happens? What should happen? Be specific.
2. Exact Console Error: Copy the full message exactly. Include the file if shown.
3. Relevant Code Only: Paste only the lines involved. Remove anything unrelated.
4. Checks Already Tried: List what you've checked or changed. This prevents repeated suggestions.
5. Ask for Explanation: Ask for likely causes to test and why. Do not ask for a full fix or new project.

Possible Causes to Test

- Selector may be wrong: The element might not exist, or the class/id is different.
- Condition logic may be off: The condition may never be met or uses the wrong operation.
- Event may not be connected: This element might be on the wrong element or not attached.
- Data value may be unexpected: Input might be empty, null, or a string instead of a number.
- CSS may be hiding the result: The container element has a hidden attribute or is not visible.

Test these in order. Retest after each change.

Good AI Debugging Prompt

23

Debugging Report Pattern

Debugging report pattern:

Issue:

- ▶ what was wrong?

Evidence:

- ▶ how did you identify it?

Fix:

- ▶ what did you change?

Verification:

- ▶ how do you know it works now?

24



Debugging Report

Name: _____ Date: _____ Assignment/Task: _____

- Issue:**
What was wrong?
(Write 1-2 sentences.)
- Evidence:**
How did you identify it?
(Console messages, console log output, browser behavior, etc./See checked.)
- Fix:**
What did you change?
(Name the file, line number, and change.)
- Verification:**
How do you know it works now?
(List result, expected behavior, no unexpected console error.)

★ Fixed code alone is not the whole assignment. Your reasoning, evidence, and verification matter.

Debugging Report Pattern

25



Lab Refinement

Verified fixes

Your goal:

- ▶ fix at least 2 confirmed issues completely
- ▶ use console output or inspection as evidence
- ▶ verify the behavior after each fix
- ▶ document issue, evidence, fix, and verification

26



Evidence & Reflection

Evidence:

- ▶ updated HTML, CSS, and JS
- ▶ site runs with fixes applied
- ▶ at least two issues fixed correctly
- ▶ debugging report included

Reflection

- ▶ Explain how your problem-solving changed

27



How To Read Next Week's Material

Required:

- ▶ read functions as a way to organize behavior
- ▶ notice scope, dot notation, and document as a built-in object

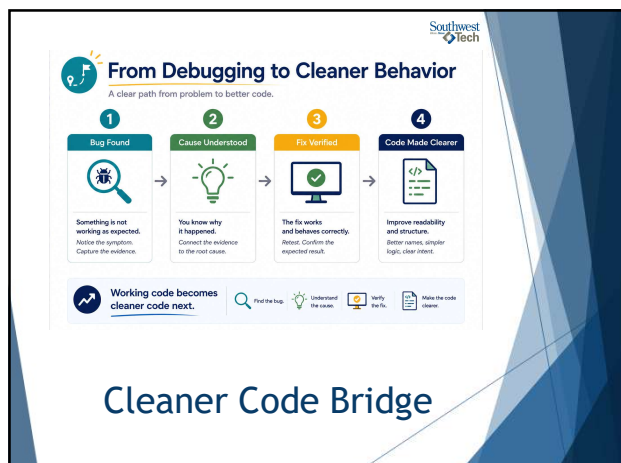
Reference:

- ▶ objects and built-in methods are there when you need them
- ▶ do not memorize every method

Before next time:

- ▶ Bring one piece of interaction code that could be made clearer.

28



29



Closing Success Check

This week:

- ▶ problems became evidence
- ▶ console messages became clues
- ▶ fixes were verified
- ▶ reports explained the process

Next:

- ▶ working code becomes cleaner code.

30



31